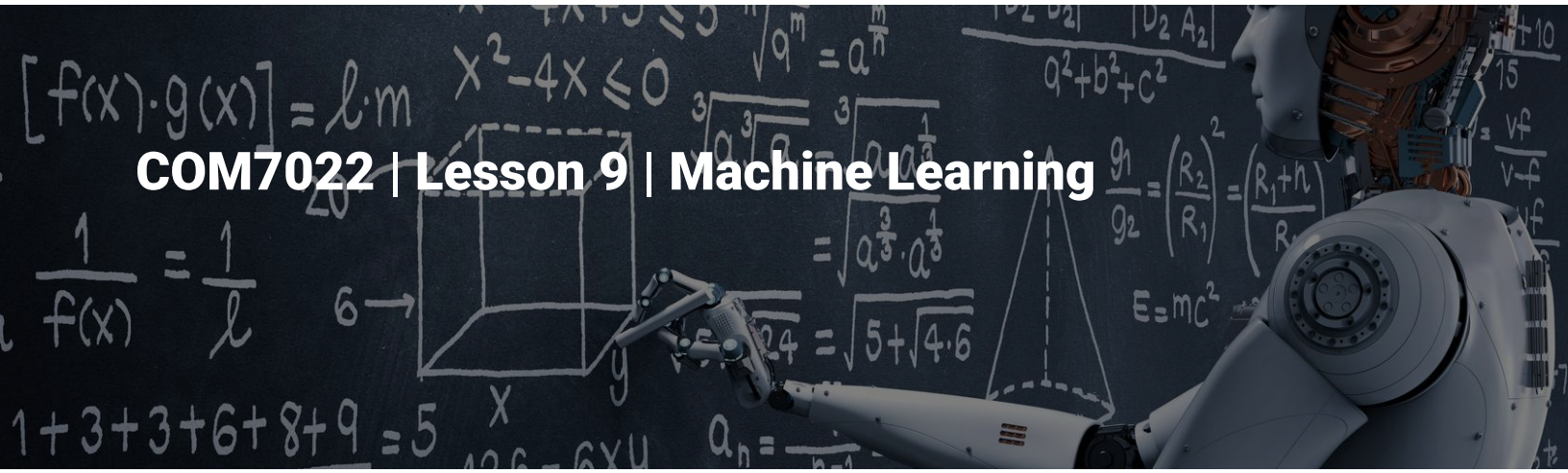


COM7022 | Lesson 9 | Machine Learning

Machine Learning (COM7022)

Lesson 9: Hierarchical Clustering

Cover image by Phonlamai Photo from Shutterstock.com

- Lesson introduction
- Topic 1: Hierarchical Clustering
- Topic 2: Agglomerative Method
- Topic 3: Divisive Method
- Topic 4: Full Model
- Knowledge check
- Lesson summary

Reading List

Lesson introduction

Welcome to Lesson 9

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 9.

Within this week's lesson we shall be discussing and demonstrating, through many examples, Hierarchal clustering techniques. We shall begin the lesson by giving a short, coded example of what one can expect to see when engaged in such a study. When engaged in hierarchical clustering there are two methods that can be used, dependent upon the study, these methods are Divisive and Agglomerative: both of which will be demonstrated.

We have a lot to get through within this first topic, so let us dive straight in.



Computing

At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

By the end of this week's lesson you will be able to:

- Evaluate what hierarchical clustering is, both from a theory perspective and practicality.
- Critically evaluate when to use Agglomerative clustering and the methods adopted to such an investigation.
- Identify what Divisive clustering is and when to ideally adopt such an approach.
- Reflect on what is included when engaged in a full hierarchical clustering investigation.

Lesson 9 aligns to Learning Objectives 2 and 3:

- **Select and apply appropriate machine learning techniques for a given scenario.**
- **Critically evaluate the results from the application of Machine Learning techniques and propose improvements.**



Computing

At Arden University

We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.

Continue to Topic 1

Topic 1: Hierarchical Clustering



Image from Articulate Rise 360 Content Library.

Hierarchical clustering is a popular method within the data mining world which is used for grouping similar and different objects within a data set together, as Figure 9.01 shows below.

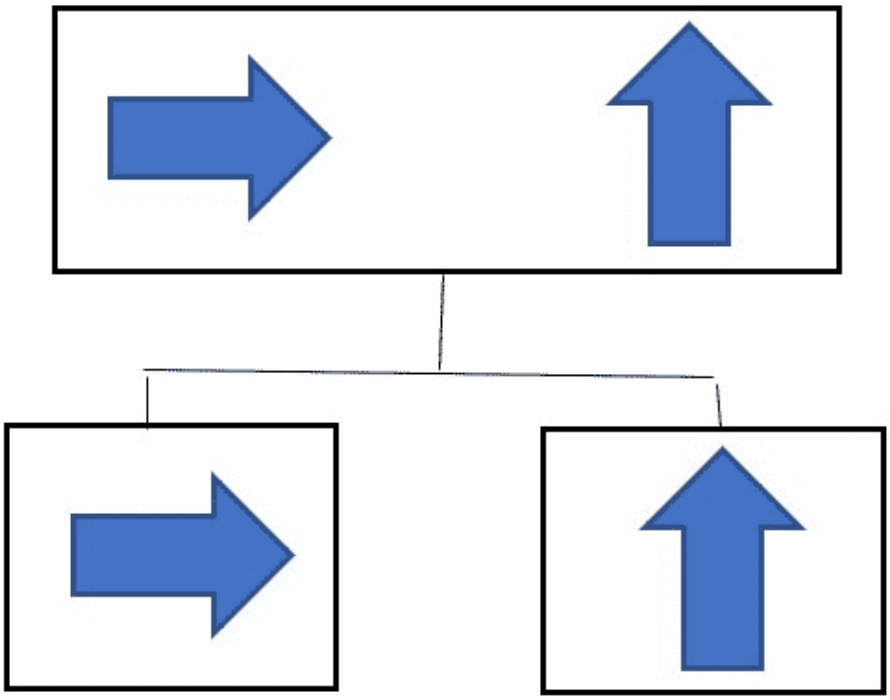


Figure 9.01. Basic visualisation of Hierarchical clustering by Arden University. (Accessible list).

As we shall when progressing through this topic, using this particular type of clustering method produces a hierarchical tree, commonly referred to as a Dendrogram.

The reason why this method is so popular amongst data analysts is that there is no requirement to specify the number of clusters from the outset, as the Dendrogram algorithm does this automatically and the result makes the initial analysis relatively easy to understand.

As can be expected from this clustering method, there are many different real-world applications in which it is applied: business, where customers are separated according to spending habits, or company employees, based upon wages or days taken on sick leave. There is image processing, where handwriting characters are grouped in text recognition. You have probably seen this many times when using an electric note pad when signing for a delivery. Then there is also information retrieval from previous search results, based upon past queries.

Therefore, Hierarchical clustering is an unsupervised clustering technique that groups data points together which share similar characteristics. The clusters are arranging in a top-down manner, thus similar clusters are grouped together and arranged in a hierarchical manner. This particular clustering techniques can be subdivided into two groups, Agglomerative and Divisive. More on these subdivisions as the lesson develops.

As a comparison, there is also non-hierarchical clustering, which involves the creations of new clusters when merging or splitting and do not follow the same hierarchical graphical concept. An ideal example of non-hierarchical clustering is K-Means, which we addressed in the previous lesson. As a reminder, K-Means the splitting occur to ensure overlapping groups have no hierarchical relationships between them. To aid in the clear distinction between Hierarchical and Non-Hierarchical clustering methods, as brief table has been included below for quick reference.

Hierarchical	Non-Hierarchical
Creates clusters in a pre-defined order from top to bottom	Creates new clusters by merging / splitting clusters in a non-hierarchical manner
Less reliable	More reliable than Hierarchical clustering, generally
Slower to perform	Faster algorithmic calculations

Hierarchical	Non-Hierarchical
Sensitive to high level of errors	No as sensitive to errors
Easier to read and interpretate results	Can be difficult to interpretate results
Can be prone to instability	A relatively stable method

Below in Figure 9.02 is a very brief example of a Hierarchical clustering algorithm working, based upon a randomly generated data set so that it is easily relatable to the theory just discussed. A more in-depth example will be given at the end of this lesson.

Example

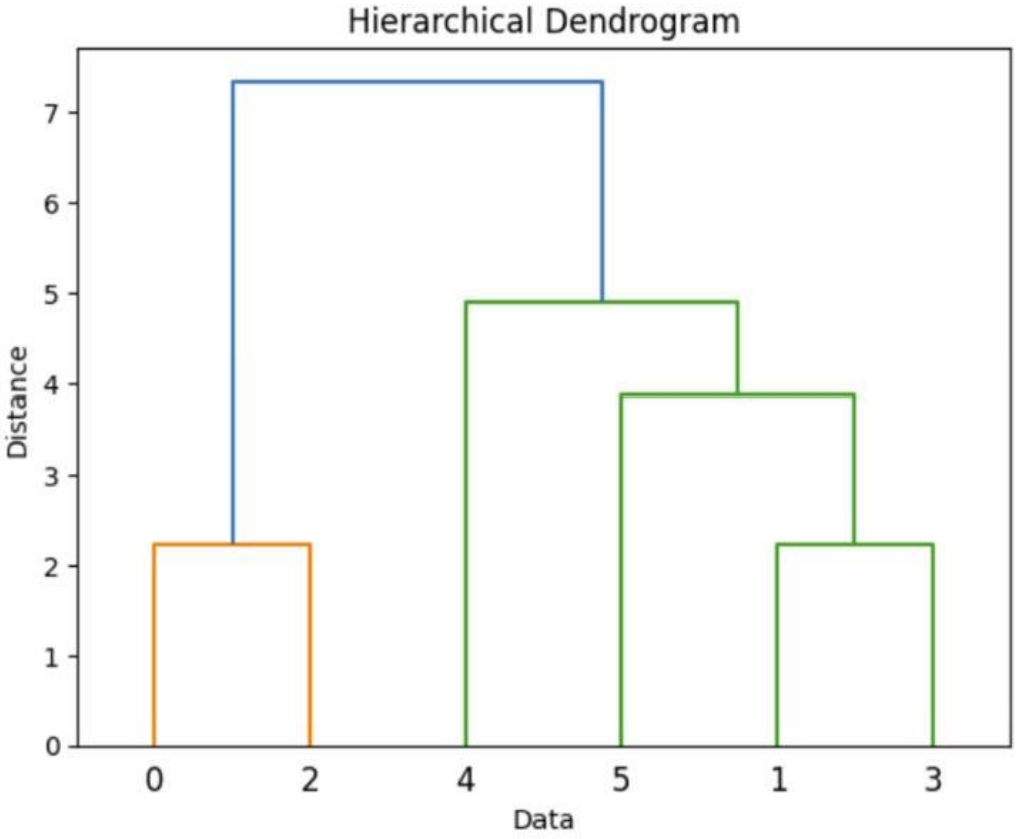


Figure 9.02. Brief introductory model using Hierarchical clustering in Google Colab by Arden University.

```python
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Generating a random dataset
X = np.array([[2, 1], [3, 6], [4, 2],
      [2, 4], [6, 6], [1, 8]])

# Calculating a hierarchical cluster
Z = linkage(X, 'ward')

# Plottting a dendrogram to visualise the cluster
dendrogram(Z)

plt.title('Hierarchical Dendrogram')
plt.xlabel('Data')
plt.ylabel('Distance')
plt.show()
```

Figure 9.02a. Brief introductory model using Hierarchical clustering in Google Colab by Arden University. (Accessible code).

You are encouraged to use the above example in which to practice. Either add/remove data points from the data set to visually see the difference this makes to the results.

Continue

Topic 1 - Activity 1

Hierarchical clustering algorithm



Create a Hierarchical clustering algorithm from 1000 randomly generated data points, which produces an associated Dendrogram. How could you improve the results of the algorithm?

Post your results and improvement resolution within the discussion forum for further analysis by the tutor and peers

Continue

Topic 1 - Activity 1

Reading



- Chapter 1: Graph Theory (pp. 21-62) from Linear Algebra with Machine Learning and Data by Arangala.
- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 4: The Measure of All Things – Statistics (pp. 179-216) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 1 SUMMARY

Well done, you have completed Topic 1.

Within this topic we have given a description of what hierarchical clustering is and when we would use such an approach, i.e., within an unsupervised learning study.

We have also discussed the differences between hierarchical and non-hierarchical clustering, plus run through a simple coded example.

Hopefully you are now well prepared to take the next step within this clustering ideology in the next topic, i.e., Agglomerative clustering

See you there.

Continue to Topic 2

Topic 2: Agglomerative Method

Image from Articulate Rise 360 Content Library.

There are two main types of Hierarchical clustering: Agglomerative and Divisive. For this particular topic we shall solely focus on Agglomerative, as it is the easier of the two, but you are encouraged to research the Divisive method so that you can make the comparison much more definitively and gain a much deeper understanding of the topic in general, before we get onto it within the following topic.

Very generally speaking, the Agglomerative method takes each object in the data set to be a single cluster. All the clusters are then merged together in a step-by-step manner, until only one cluster remains. In summary:

- A proximity matrix is defined using a stated distance metric, usually Euclidean distance, but there are others.
- Each data point is assigned to a cluster.
- The clusters are then merged, based upon the defined distance metric, for similarity between the clusters.
- Update the proximity matrix.
- Repeat steps three and four above until only one cluster remains.

Say for example we have six data points within our data set, as shown in Figure 9.03. The initial step is to calculate proximity values of the individual data points as individual cluster.

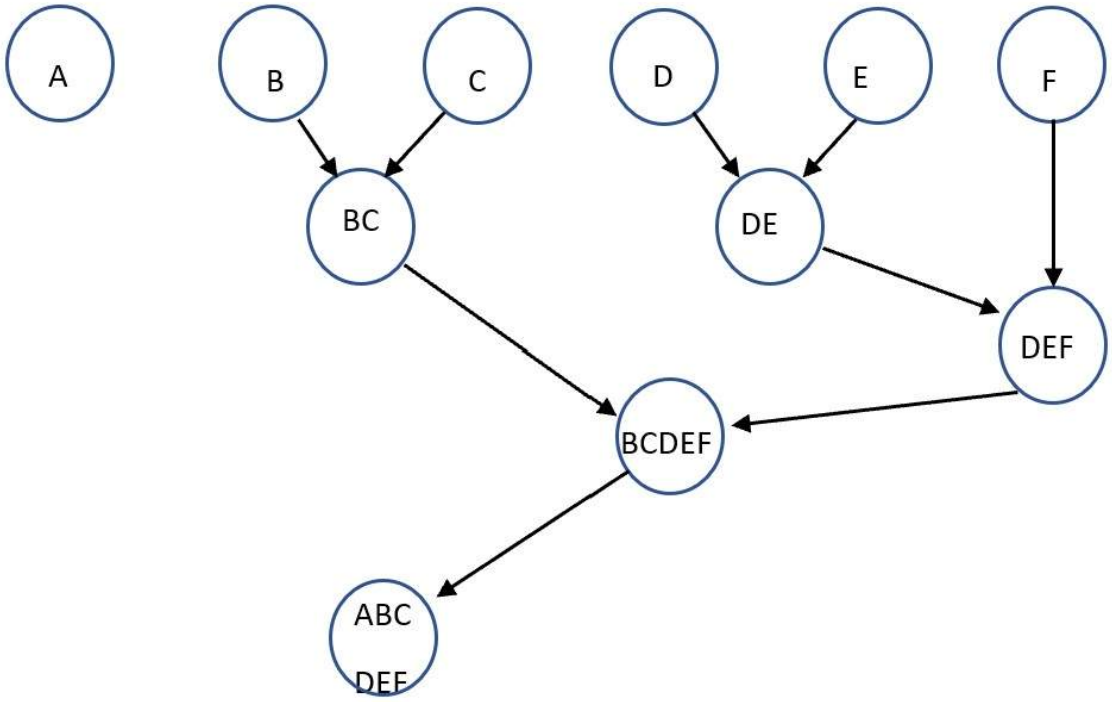


Figure 9.03. Agglomerative Clustering by Arden University. (Accessible list).

In the next step two similar clusters are bought together that share some characteristics and form on single cluster. Thus, this now leaves clusters A, BC, DC and F. Once the initial clustering has taken place, the next stage of the algorithm the proximity is again calculated of the newly formed clusters, which in turn will result in clusters A, BC and DEF.

In the final two steps, the cluster proximity is again calculated, which will bring cluster A, B, C, D and E and finally, all the clusters have merged, as shown in Figure 9.03. The results can either be shown in Figure 9.04 as a Dendrogram from the original data.

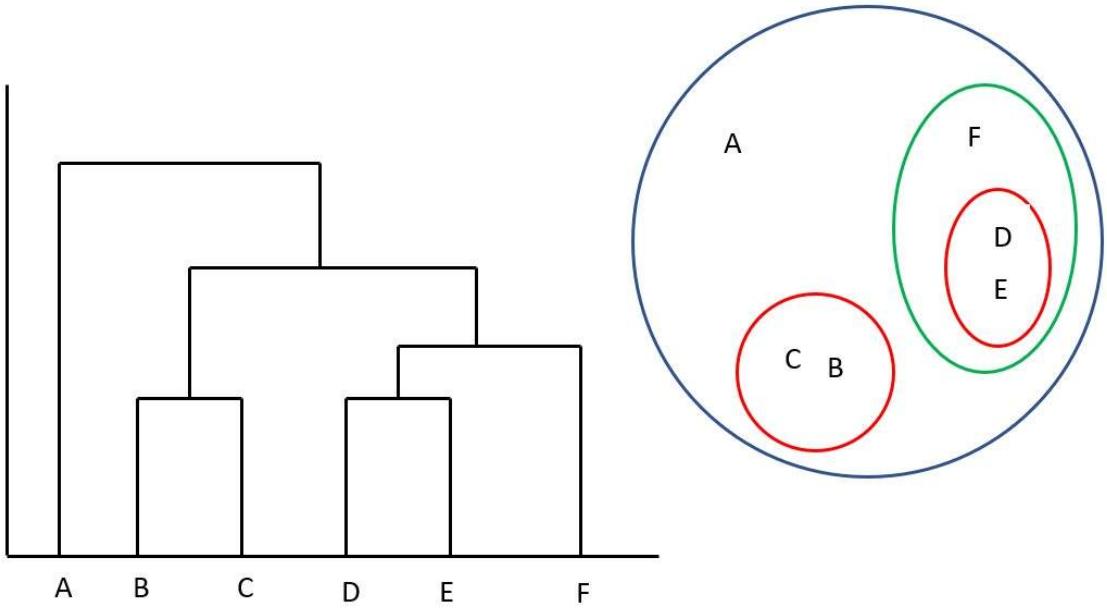


Figure 9.04. Graphical representation of a moving from a proximity cluster to hierarchical cluster by Arden University.

The main question when performing Hierarchical clustering is how do we determine the distances between the clusters and update the proximity matrix? Through research over many years, there are multiple different ways in which this has been developed using linkage. Each linkage has its own advantages and disadvantage, therefore giving the analysist multiple options. We shall briefly discuss four of the more common linkages in the following topic.

To demonstrate the agglomerative method, shown below in Figure 9.05, let us give a brief coded example based upon the theory given at the beginning of this topic and in Figure 9.03.

```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering

# Generating a random dataset
X = np.array([[1, 2], [1, 4], [1, 0],
      [4, 2], [4, 4], [4, 0]])

# Calculating a hierarchical cluster
clustering = AgglomerativeClustering(n_clusters=2).fit(X)

print(clustering.labels_)
```

```
[1 1 1 0 0 0]
```

Figure 9.05. Coded example based upon the information given in Figure 9.03 in Google Colab by Arden University. (Accessible code).

Again, you are encouraged to practise this for yourself, altering the allocated data points.

Below in Figure 9.06 is a second, more extended example of the Agglomerative clustering method using a randomly generated data set of fifty data points and four features. The linkage we shall be using in this example will be the complete linkage, which will be explained in more detail in the following topic.

```python
# Importing required libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

# Creating a random dataset of 50 samples with four features
np.random.seed(0)
x = np.random.randn(50, 4)

# Application of the complete linkage method
z = linkage(x, method='complete')

# Creating a dendrogram
plt.figure(figsize=(12, 4))
plt.title('Agglomerative Dendrogram')
plt.xlabel('index')
plt.ylabel('Distance')
dendrogram(z)
plt.show()
```

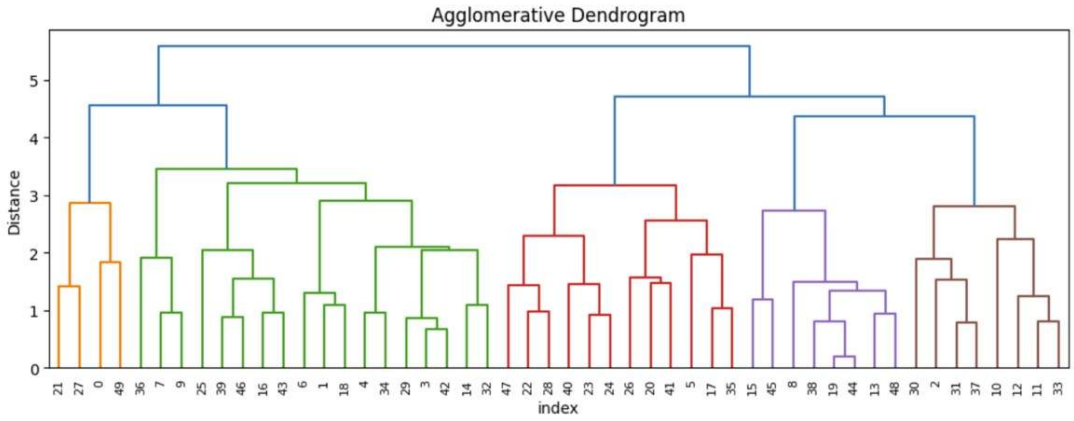
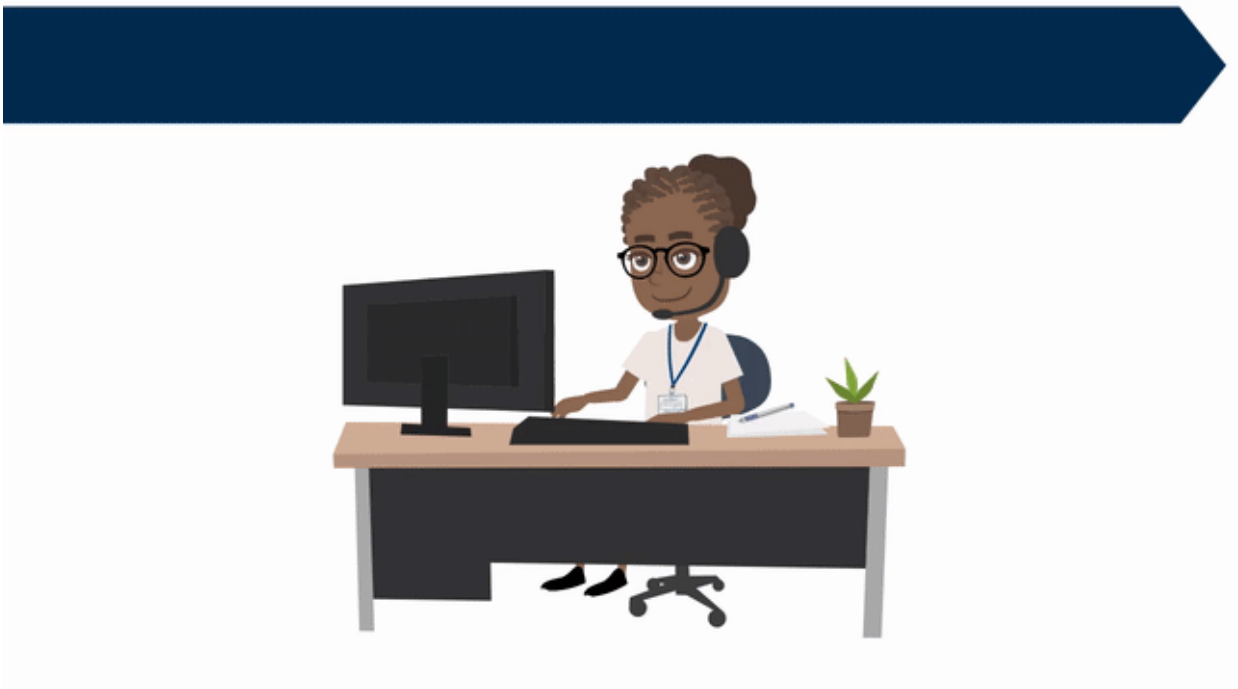


Figure 9.06. Extended example of agglomerative clustering 3 in Google Colab by Arden University. (Accessible code).

Continue

Topic 2 - Activity 1

Hierarchical clustering algorithm



1. Create a Hierarchical clustering algorithm from 1000 randomly generated data points, which produces an associated Dendrogram. Use Wards method. Post your result in the discussion forum for further analysis by the tutor and peers.
2. Create a Hierarchical clustering algorithm from 2000 randomly generated data points, which produces an associated Dendrogram. Use Wards method and vary the linkages.

How could you improve the results of the algorithm? Post your result in the discussion forum for further analysis by the tutor and your peers.

Continue

Topic 2 - Activity 2

Reading

- Chapter 2: Stochastic Processes (pp. 63-104) from Linear Algebra with Machine Learning and Data by Arangala.
- Chapter 2: Anaconda, Spyder and the Libraries NumPy, Matplotlib and SymPy (pp. 41-52) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 4: The Measure of All Things – Statistics (pp. 179-216) from Statistics and Data Visualisation with Python by Rogel-Salazar.



Continue

TOPIC REVIEW

Well done, you have completed Topic 2.

As you are now aware and generally speaking, the Agglomerative clustering method takes each object in the data set to be a single cluster. All the clusters are then merged in a step-by-step manner until only one cluster remains, and this is achieved through proximity values of the individual data points.

Now that we have firm grip on what Agglomerative clustering is, we can look at the more complex, alternative hierarchical clustering technique in the following topic, i.e., the Divisive method.

See you there.

Continue to Topic 3

Topic 3: Divisive Method



Image from Articulate Rise 360 Content Library.

In comparison to the Hierarchical clustering technique, the Divisive method is not as widely used. This particular method, also known as the top-down approach, is somewhat opposite to the Agglomerative method. Here all of the points are considered as one, single cluster. The algorithm then, through multiple iterations, separates the data points based upon non-similarity. Every data point with is derived from this method is then classed as a cluster in its own right. As can no doubt be guessed, with Divisive clustering, the algorithm can give many (n) clusters. So in short, we are taking one big cluster, encompassing all data points and then dividing into n clusters, hence the terminology Divisive.

If we revert back to determining the calculation of the similarities between clusters for a moment, we can explore this a little further. As we have already mentioned in the previous topic, being able to determine the similarities/differences between data characteristics id of vital importance. As well as the complete and single linkages previously mentioned, there are also Minimum, Maximum, Group and Average linkages, to which we shall discuss separately below.

Single linkage: also referred to as Minimum can be defined as the similarity of two points, P_i and P_j from two associated clusters A_i and Aj. Mathematically, this can be defined as

$$Sim(A_i, A_j) = Min\ Sim\ (P_i, P_j) \ni P_i \in A_i\ \&\ P_j \in A_j$$

In short, a point is found in the first cluster, which is nearest to a point in another cluster, then the distance between the two is calculated. This can be seen graphically below in Figure 9.07

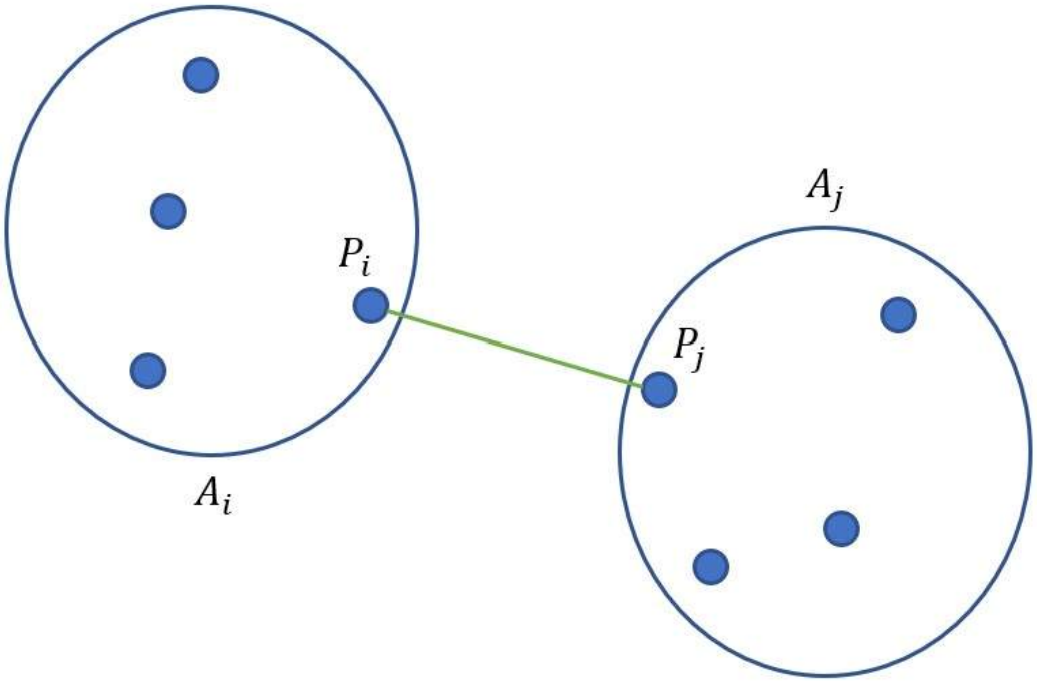


Figure 9.07. Graphical representation of point to point of the divisive clustering technique by Arden University.

Complete Linkage: also referred to as Maximum is in essence the opposite to single linkage. Where we addressed the closest, minimum data points in the clusters, here we address the maximum distanced data points, as Figure 9.08 shows. The reason why this approach works well is quite often there is noise between the clusters and the maximum techniques deals with this quite well. The downfall to this however is that this approach tends to break up large clusters, which can be unnecessary at times. Mathematically written we will have:

$$Sim(A_i, A_j) = Max\ Sim\ (P_i, P_j)\ \ni\ P_i \in A_i\ \&\ P_j \in A_j$$

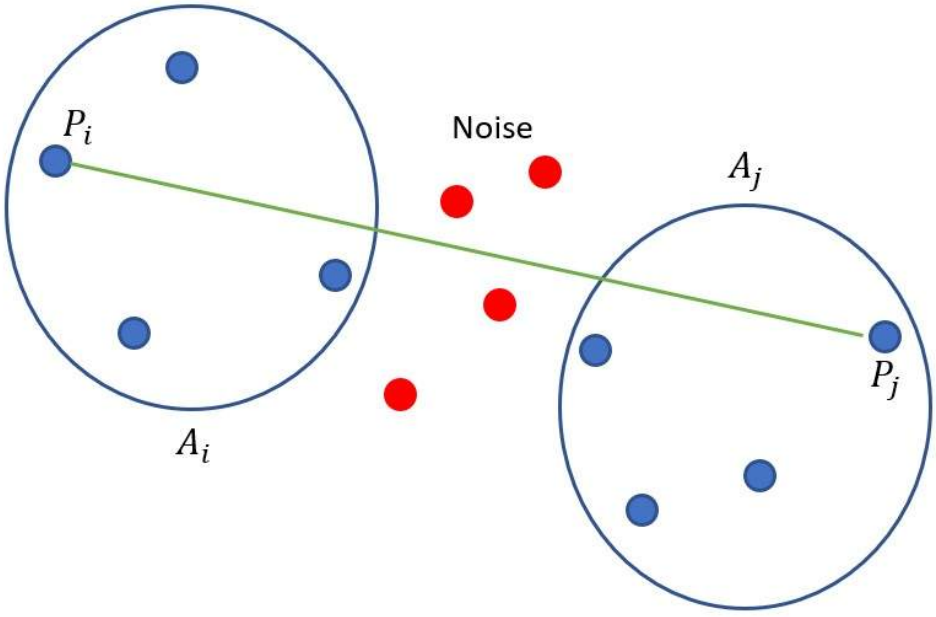


Figure 9.08. Mathematical representation of Divisive Clustering by Arden University.

Group averaging: takes all the pairs of points and yes you have guessed it, determines their similarities and calculates the average of their similarities, as the mathematical definition below states and Figure 9.09 shows. This approach works quite well when there is a lot of noise between the clusters. However, the groups average methos is slightly bias towards global clusters, due to the increased influence of the centroids within each cluster. Averaging anything is always the most unbiased way of determining results over the entire data set.

$$Sim(A_i,\ A_j) = \sum Sim \frac{P_i, P_j}{\lfloor A_i \rfloor * \lfloor A_j \rfloor},\ where\ P_i \in A_i\ \&\ P_j\ \in\ A_j$$

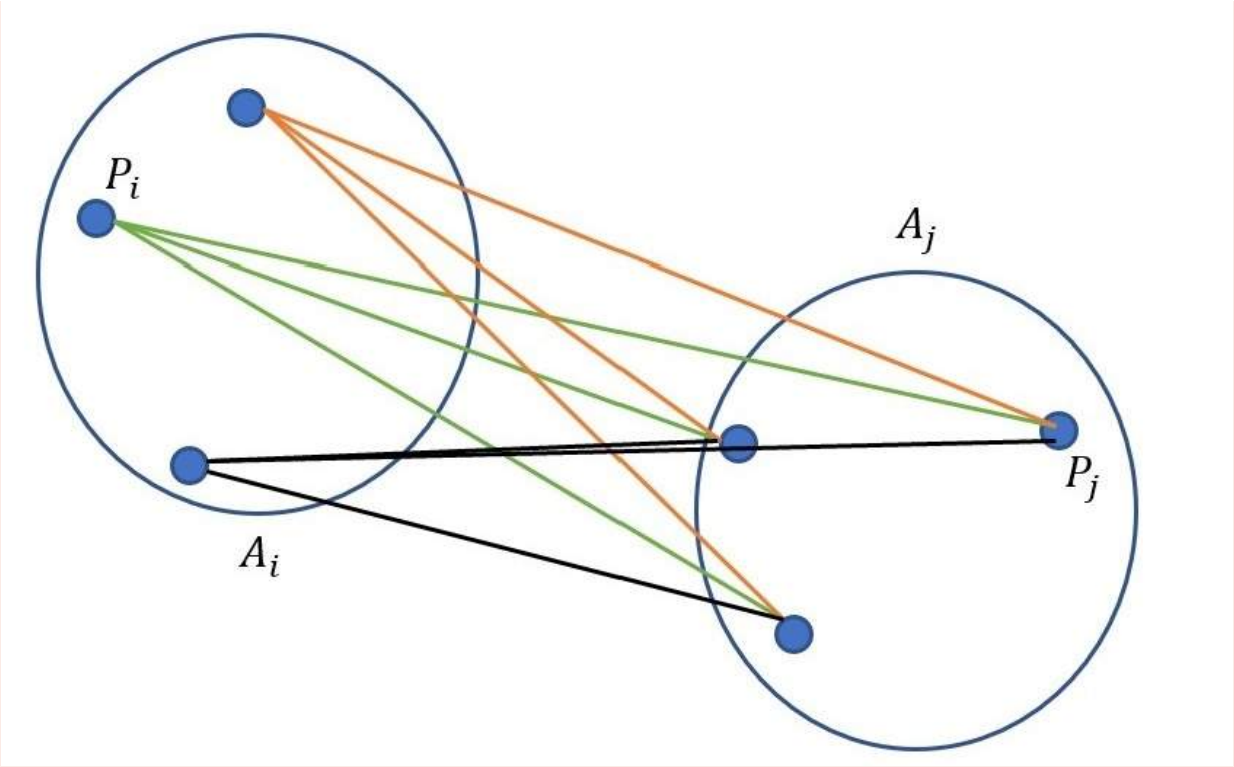


Figure 9.09. Group averaging with Divisive clustering by Arden University.

Wards method: is the same as the Group approach just mentioned, except we calculate the sum of the squares of the distances of the associated data points, or in other words, the variances. By approaching the problem in this manner, the technique is good when there is a lot of noise between the clusters. However, by adopting the variances and inherent Sum of Square calculations, we minimise the possibility of outliers, which I am sure you will agree can only be a good thing. As such, the Ward method, demonstrated in Figure 9.10, is the preferred method within Agglomerative Hierarchical clustering.

Example

$$Sim(A_i,\ A_j) = \sum Sim \frac{(dis(P_i, P_j))^2}{\lfloor A_i \rfloor * \lfloor A_j \rfloor},\ where\ P_i \in A_i\ \&\ P_j \in A_j$$

```python
# Import libraries
from scipy.cluster.hierarchy import dendrogram, linkage
import numpy as np
import matplotlib.pyplot as plt

# Generate sample data
np.random.seed(0)
X = np.random.randn(50, 4)

# Perform divisive clustering
Z = linkage(X, method='ward', metric='euclidean')

# Generating dendrogram
plt.figure(figsize=(12, 4))
plt.title('Agglomerative Clustering')
plt.xlabel('index')
plt.ylabel('Distance')
dendrogram(Z)
plt.show()
```

Figure 9.10a. Agglomerative clustering using the Ward method in Google Colab by Arden University. (Accessible code).

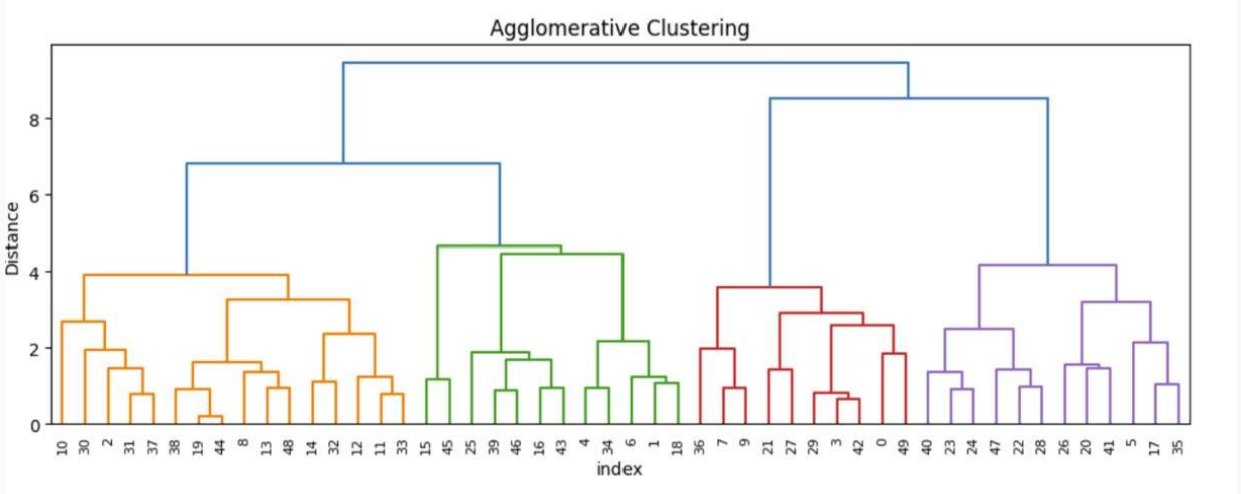


Figure 9.10b. Agglomerative clustering using the Ward method in Google Colab by Arden University. (Accessible code).

Continue

Topic 3 - Activity 1

Reading



- Chapter 2: Stochastic Processes (pp. 63-104) from Linear Algebra with Machine Learning and Data by Arangala.
- Chapter 3: Jupyter Notebooks and Google Colab (pp. 53-65) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 5: Definitely Maybe: Probability and Distributions (pp. 217-304) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC REVIEW

Well done, you have completed Topic 3.

In comparison to the Agglomerative technique, the divisive method is known as the top-down approach, which is opposite to the Agglomerative method. In the divisive method all of the points are considered as one, single cluster and as we now know, the algorithm performs multiple iterations and separates the data points based upon non-similarity. As we saw in the topic, every data point with is derived from this method is then classed as a cluster in its own right.

Hopefully you now fully understand the difference between Agglomerative and divisive clustering methods, but if not, then please re-visit both topics so that you have a full and rounded knowledge on the ideologies. Having now completed Topic 3, we are well laced to engage in a full coded example on hierarchical clustering in the final topic.

See you there.

Continue to Topic 4

Topic 4: Full Model



Image from Articulate Rise 360 Content Library.

We shall now run through an example using all four linkage methods detailed above, so that we can determine and more importantly compare the differing results. The example will use a randomly created array in Python, linkage methods will then be defined and results given.

First, we import the libraries from various locations and define the random array.



Figure 9.11. Importing libraries, functions and creating the array in Google Colab by Arden University. (Accessible code).

The relevant libraries now imported, including the seaborn library, so that we can style the plot better, we can plot the array using a scatter plot. The code and resulting plot are shown in Figure 9.12 below.

```
plt.figure(figsize=(6, 6))

plt.scatter(X1[:,0], X1[:,1], c='r')

# Create numbered labels for each point
for i in range(X1.shape[0]):
    plt.annotate(str(i), xy=(X1[i,0], X1[i,1]), xytext=(3, 3), textcoords='offset points')

plt.xlabel('x coordinate')
plt.ylabel('y coordinate')
plt.title('Scatter Plot of the data')
plt.xlim([0,10]), plt.ylim([0,10])
plt.xticks(range(10)), plt.yticks(range(10))
plt.grid()

plt.show()
```

Figure 9.12. Plotting the data points of the array in Google Colab by Arden University. (Accessible code).

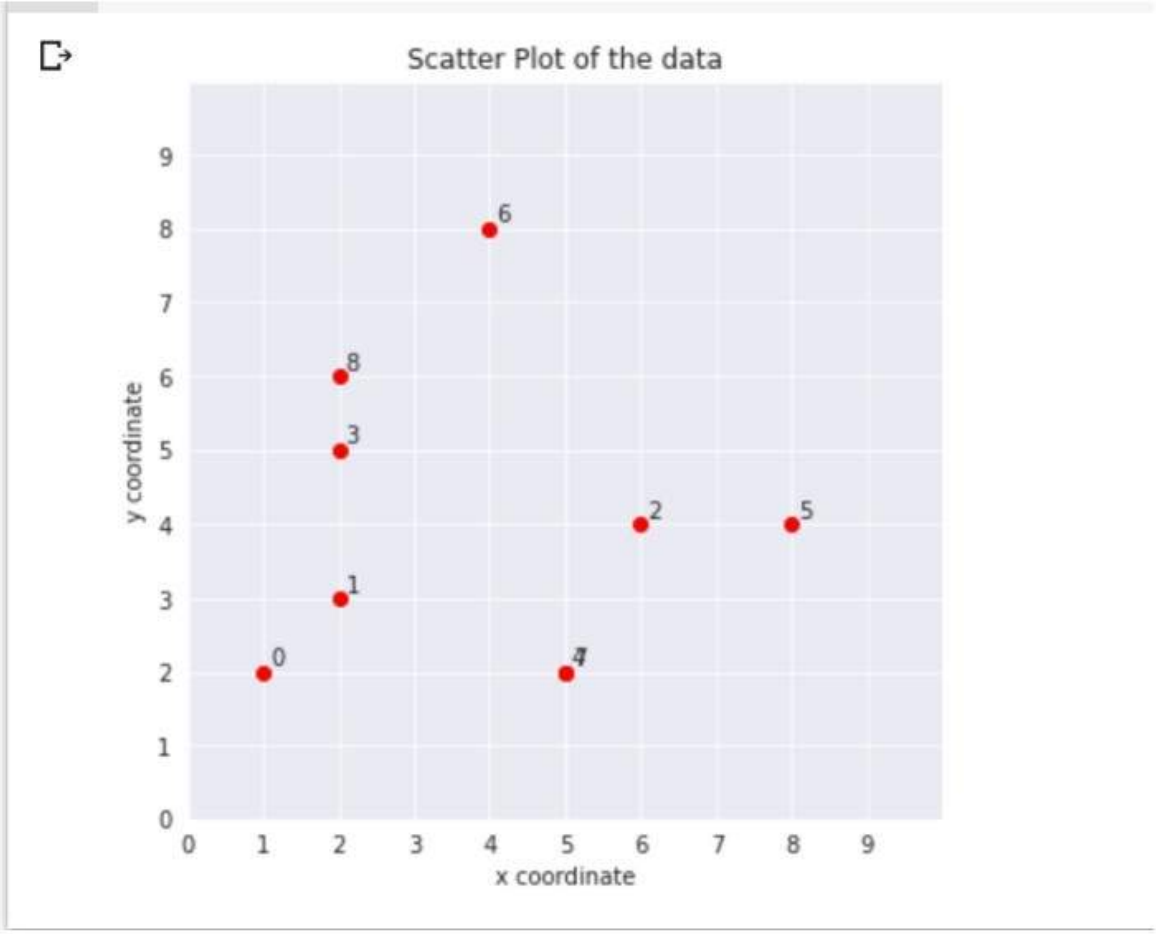


Figure 9.12. Plotting the data points of the array in Google Colab by Arden University. (Accessible code).

So that we can now begin to build the resulting Dendrograms using the linkage methods detailed previously, we need to import the **scipy** library, as this library includes the **linkage** and **Dendrogram** functions. If you notice, four new variables have been created (Zn) so that we can assign each linkage methods to include the same Euclidean distance measure, again, this was briefly mentioned at the beginning of this topic. As seen in Figure 9.13 below.

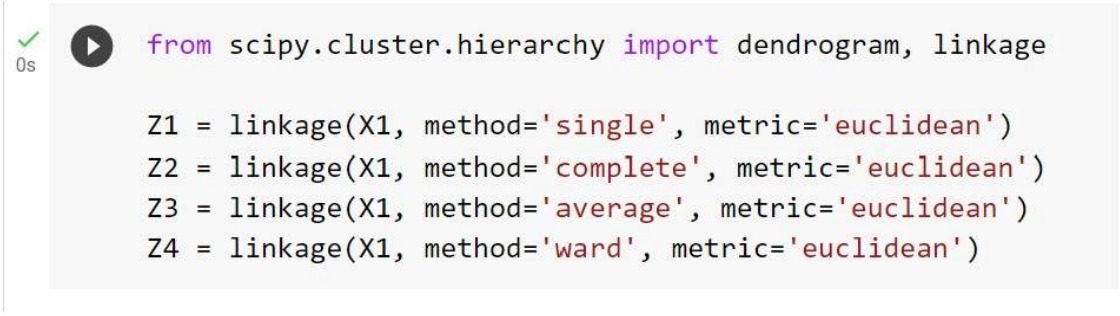


Figure 9.13. Importing the required functions from the scipy library and defining different variables in Google Colab by Arden University. (Accessible code).

The different linkages now being assigned, we can pass the variables through Matplotlib, which we imported earlier, so that we can view the resulting Dendrograms. The code and results are contained in Figure 9.14 below.

```
plt.figure(figsize=(15, 10))
plt.subplot(2,2,1), dendrogram(Z1), plt.title('Single')
plt.subplot(2,2,2), dendrogram(Z2), plt.title('Complete')
plt.subplot(2,2,3), dendrogram(Z3), plt.title('Average')
plt.subplot(2,2,4), dendrogram(Z4), plt.title('Ward')
plt.show()
```

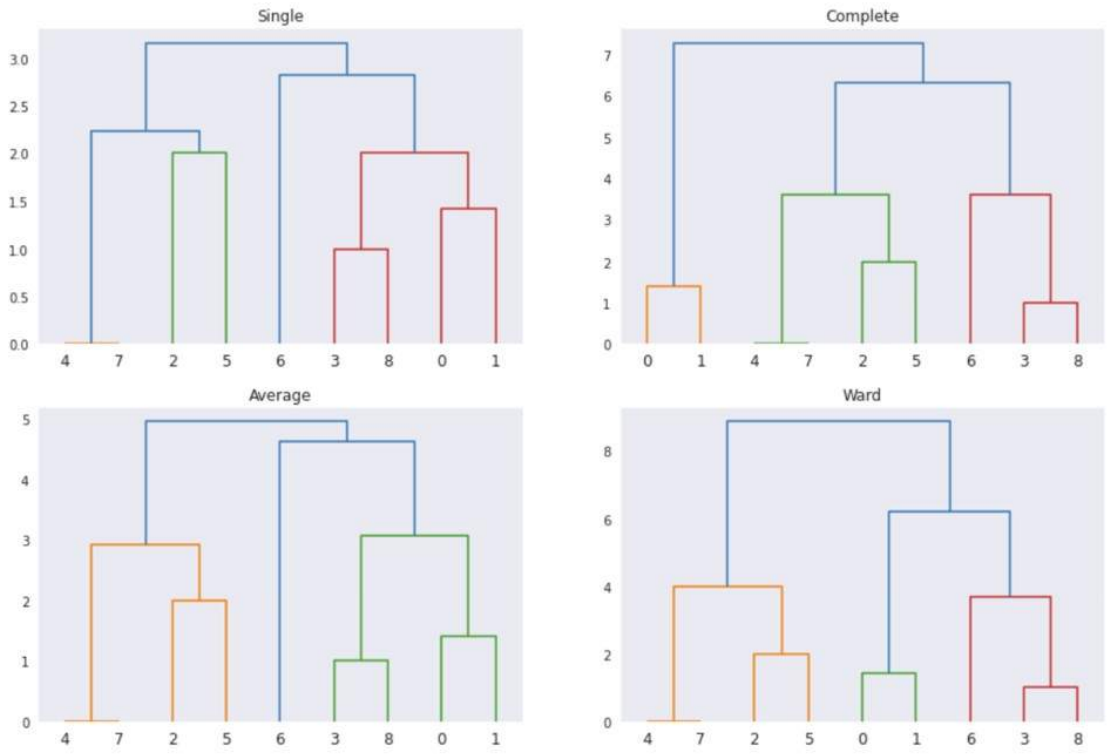


Figure 9.14. Dendrogram results for each linkage method in Google Colab by Arden University. (Accessible code).

As you can see from Figure 9.14, each linkage method provides different results, based upon each inherent feature, again, which we briefly detailed earlier on in the topic. So that we can study the Ward linkage method, we import the **fcluster** function from the **scipy** library once more. The results are shown in Figure 9.15.

```
from scipy.cluster.hierarchy import fcluster

f1 = fcluster(Z4, 2, criterion='maxclust')

print(f"Clusters: {f1}")
```

```
Clusters: [2 2 1 2 1 1 2 1 2]
```

Figure 9.15. Assigned data points using the Ward linkage method in Google Colab by Arden University. (Accessible code).

What Figure 9.15 is informing us is that the first data point in our originally defined array has been assigned to cluster 2 and so on. We now imported the **Agglomerative clustering class** from the **sklearn.cluster** library, as shown in Figure 9.16.

```
from sklearn.cluster import AgglomerativeClustering

Z1 = AgglomerativeClustering(n_clusters=2, linkage='ward')

Z1.fit_predict(X1)

print(Z1.labels_)
```

```
[0 0 1 0 1 1 0 1 0]
```

```
[8] fig, ax = plt.subplots(figsize=(6, 6))

scatter = ax.scatter(X1[:,0], X1[:,1], c=Z1.labels_, cmap='rainbow')

legend = ax.legend(*scatter.legend_elements(), title="Clusters", bbox_to_anchor=(1, 1))
ax.add_artist(legend)

plt.title('Scatter plot of clusters')

plt.show()
```

Figure 9.16. Importing the Agglomerative clustering class in Google Colab by Arden University. (Accessible code).

All that is left to do is to use the scatterplot so we can visually see the different clusters that the **Agglomerative class** has calculated. Figure 9.17 demonstrates the comparative between our original data set and the final result.

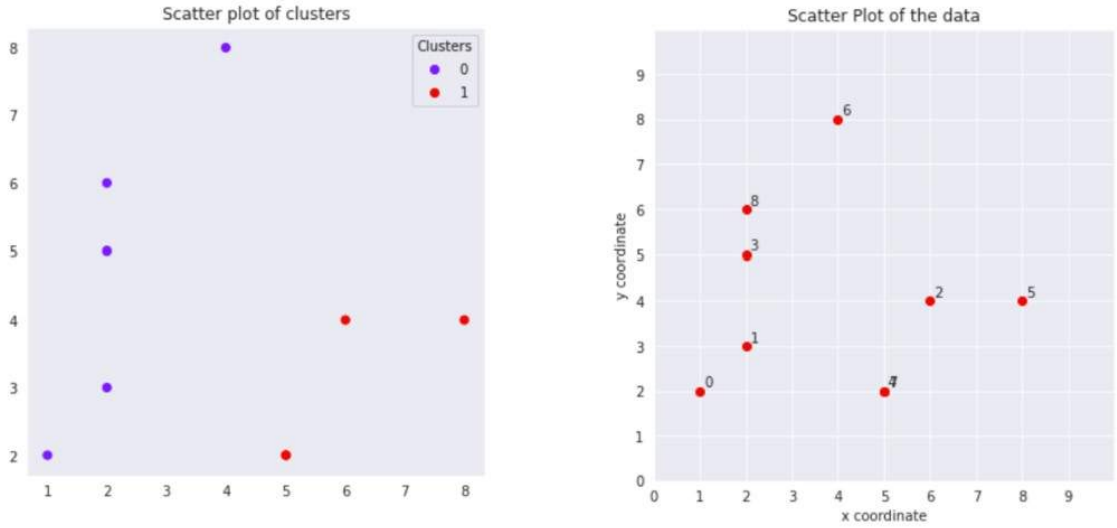


Figure 9.17. Comparative visualisation of the original data set (right) and the Agglomerative function (left) being applied in Google Colab by Arden University.

To sum up, we created a fictitious array, plotted the data points on a scatter plot and bought in several libraries which were needed for their inherent functions. We then created new variables and assigned the different linkage methods to each, all with the same distance measure, i.e. Euclidean distance. Finally, we passed the Ward result through the **sklearn Agglomerative.cluster** class to see how the method compared to our original scatter plot. This then resulted in comparative Figure 9.17.

Continue

Topic 4 - Activity 1

Hierarchical clustering

Using the example within the lesson, please feel free to change various parameters as you see fit. Increase/decrease data points, import your own data etc. Use this activity to become increasingly familiar with Hierarchical clustering as an unsupervised machine learning algorithm. Once you have completed your practice model, post your results in the group's forum for further discussions.



Continue

Topic 4 - Activity 2

Reading



- Chapter 3: SVD and PCA (pp. 105-136) from Linear Algebra with Machine Learning and Data by Arangala.
- Chapter 3: Jupyter Notebooks and Google Colab (pp. 53-65) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 5: Definitely Maybe: Probability and Distributions (pp. 217-304) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 4 SUMMARY

Well done, you have completed Topic 4.

Topic 4 was the culmination of everything that you had learned within all previous topic on hierarchical clustering. The example given is involved, of this there is no doubt; however, this example can be adopted to any real-world data set, if the correct steps are taken. No doubt as you continue with your career you will visit this example as a base and build from there.

All topics within Lesson 9 have now been concluded.

Continue to Knowledge check

Knowledge check

Select the correct answer to the following questions.

1 of 2: What does the Ward linkage do?

- The Ward approach utilises the minimum distances of the clusters.
- The Ward approach utilises the average distances of the clusters.
- The Ward approach utilises the variances of the clusters, rather than measuring the distances directly.
- The Ward approach puts the clusters in hospital.

SUBMIT

2 of 2: In DBSCAN, what will increasing the distance around a data point do?

- ◯ It will usually reduce the resulting number of clusters
- ◯ It will usually increase the resulting number of clusters
- ◯ Nothing at all

SUBMIT

Continue to Lesson summary

Lesson summary

LESSON SUMMARY

Congratulations, you have completed Lesson 9.

Lesson 9 has been solely concerned with Hierarchical clustering methods. Within the first topic we gave a description of what hierarchical clustering is and when such an approach would be used, i.e., within an unsupervised learning study. We also discussed the differences between hierarchical and non-hierarchical clustering, plus ran through a simple coded example. Within Topic 2 we ran through the Agglomerative clustering method, where it takes each object in the data set to be a single cluster. All the clusters are then merged in a step-by-step manner until only one cluster remains, and this is achieved through proximity values of the individual data points.

In Topic 3, as a comparison to the Agglomerative technique, we looked at the Divisive method, also known as the top-down approach, opposite to the Agglomerative method, and in the final topic we culminated everything that you had learned in a full example. Now that we have concluded all topics, hopefully you have a much better understanding of Hierarchical clustering and the two different methods that can be used.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST

References

Cited sources that do not appear in your reading list:

The Economic Times, (2017). Big data analytics to become $16 billion industry by 2025. *The Economic Times* [online], 2 July 2017.